

Multi-Role Hiring Automation System

n8n Automation Developer Assignment

Walkthrough & Architecture Document

Muhammad Immad

Submitted: May 2026.

1. Project Overview

This document describes the architecture, design decisions, and implementation details of the Multi-Role Hiring Automation System built in n8n. The system automates the entire hiring pipeline for two roles, Software Engineer (SWE) and Business Development Manager (BDM) from application ingestion through AI screening to candidate communication.

1.1 What the System Does

- Pulls new applications from two separate Google Sheets (one per role) every 30 minutes
- Standardizes and cleans inconsistent form data into a unified format
- Downloads resume files from Google Drive and extracts content for AI analysis
- Runs role-specific AI screening using Grok (free tier)
- Stores all candidate records in a centralized Master Google Sheet
- Sends professional HTML emails and creates calendar invites on a separate hourly schedule
- Handles edge cases: empty runs, missing resumes, API failures, duplicate prevention

2. Workflow Architecture

The system is split into two independent sub-workflows running on separate schedules. This is a deliberate architectural choice to decouple processing from communication.

2.1 Sub-Workflow 1: Application Processing (Every 30 Minutes)

This workflow handles ingestion, normalization, AI screening, and storage.

Step-by-Step Flow:

1. Schedule Trigger fires every 30 minutes
2. Two parallel branches read the SWE Sheet and BDM Sheet simultaneously
3. Each branch runs a Code node to normalize raw form data into a unified schema
4. An IF node checks if there are any new applications exits gracefully if none
5. A Code node extracts the Google Drive File ID from the resume URL
6. Google Drive node downloads the resume file using OAuth2
7. A Code node prepares the resume content for the AI
8. HTTP Request node calls the Grok API with a role-specific prompt
9. A Code node parses the JSON response from Grok
10. Google Sheets node appends the full candidate record to the Master Sheet

2.2 Sub-Workflow 2: Email Sending (Every Hour)

This workflow runs independently on a 1-hour schedule to send emails to candidates who have been processed but not yet contacted.

Step-by-Step Flow:

11. Schedule Trigger fires every hour
12. Google Sheets node reads all rows from the Master Sheet where 'Email Sent' = 'No'
13. IF node checks if there are any pending candidates exits if none
14. IF node routes on classification: 'strong' goes to interview path
15. Strong: Google Calendar creates an interview event with the candidate as attendee, then Gmail sends invitation
16. Average: Gmail notifies the hiring manager for manual review
17. Weak: Gmail sends a professional, warm rejection to the candidate
18. After each email, the Master Sheet row is updated: Email Sent → 'Yes'

3. Architecture & Design Decisions

3.1 Why Two Separate Workflows?

The assignment explicitly required email sending to run on its own schedule, independent of processing. This decoupling provides several benefits:

- Resilience: if the AI API fails, emails for already-processed candidates are unaffected

- Flexibility: email schedule can be changed without touching the AI logic
- Batching: avoids rate-limiting issues by not sending all emails inline during a processing run
- Idempotency: the 'Email Sent' flag prevents any candidate from receiving duplicate emails

3.2 Why Grok?

Grok was chosen as the AI model because:

- It is completely free (1 million tokens/day on the free tier)
- No credit card required for API key generation
- Fast response times suitable for automated pipelines
- Strong instruction-following for structured JSON output

3.3 Role-Specific AI Prompts

The most critical architectural requirement was that AI evaluation must be role-specific. This was achieved by maintaining two entirely separate AI call nodes, each with a different system prompt:

- SWE prompt emphasizes: programming languages, frameworks, GitHub contributions, system design, code quality, CS fundamentals
- BDM prompt emphasizes: revenue track record, client relationships, industry knowledge, CRM tools, communication skills, sales methodology

This ensures a candidate is never judged on irrelevant criteria — a great salesperson won't be penalized for not knowing Python, and a great engineer won't be penalized for not having sales numbers.

3.4 Data Normalization Strategy

Google Forms column names are unpredictable and inconsistent. The normalization Code nodes handle this with a fallback chain:

For example, to find the candidate's name, the code tries: `row['Full Name'] || row['Name'] || row['Your Name'] || ''`

This means the workflow works regardless of how the form question was phrased. All data is standardized into a fixed schema before any downstream processing.

3.5 Master Sheet Schema

The centralized Master Google Sheet ('Candidates' tab) has these fixed columns:

Column	Purpose
Name	Candidate full name
Email	Candidate email (used as unique key for updates)
Role	Software Engineer or Business Development Manager
Submitted At	Timestamp from original form submission

Resume URL	Original Google Drive link to resume
AI Score	0-100 score assigned by Gemini
AI Classification	strong / average / weak
AI Summary	2-3 sentence evaluation summary
AI Strengths	Key strengths identified by AI
AI Concerns	Potential concerns flagged by AI
AI Recommendation	Interview recommendation text
Status	Interview / Manual Review / Rejected
Email Sent	Yes / No – prevents duplicate emails
Processed At	Timestamp of AI evaluation

4. Assumptions Made

- Google Forms are configured with 'File upload' questions, and the sheet stores Google Drive URLs (not raw files)
- The resume Drive files are accessible to the Google account used for OAuth2 authentication
- The hiring manager's email is set to 'hiring@trillesai.com' – this should be updated in the Gmail node
- Interview slots are scheduled 3 business days from the time of processing at 10:00 AM – this is configurable
- A single Google account is used for all Google services (Sheets, Drive, Gmail, Calendar) for simplicity
- The SWE form includes columns for technical skills and GitHub; the BDM form includes sales experience and LinkedIn
- Candidates who do not have a resume URL are still processed – the AI evaluates based on form field data only
- A candidate is considered 'new' if their row has no Status value or Status is 'Pending'

5. Known Limitations & Future Improvements

5.1 Current Limitations

- Resume text extraction: The current implementation downloads the file but does not perform OCR or text parsing. For a production system, a PDF parsing library (like pdf-parse) or an OCR

API would extract full resume text before sending to the AI. This is noted as a technical constraint of the n8n environment.

- Duplicate detection: The system checks 'Email Sent = No' but does not prevent the same candidate from applying twice. A deduplication check by email before appending to the master sheet would be needed.
- Google Calendar invite timing: Interview slots are hardcoded to 3 days from processing at 10 AM. A more advanced version would check calendar availability using the Google Calendar Free/Busy API.
- Error handling: If the Gemini API returns an error, the workflow falls back to a default 'average' classification rather than retrying. A proper retry mechanism would improve reliability.
- Scalability: Processing runs sequentially per candidate. For high volumes (100+ applications/run), parallel processing batches would be more efficient.

5.2 With More Time, I Would Add

- Full PDF text extraction using a Code node with the pdf-parse npm package
- An error workflow that catches failures and sends a Slack/email alert to the admin
- A deduplication check before storing each candidate
- A web-based dashboard (using n8n's webhook + a simple HTML page) to view all candidates and their scores
- Integration with an ATS (Applicant Tracking System) like Greenhouse or Lever via their APIs
- Automated follow-up emails if candidates don't respond to interview invites within 48 hours

6. Error Handling & Edge Cases

No new applications	The normalization Code node returns { skip: true } and the downstream IF node stops execution cleanly with no errors
Missing resume URL	The Drive ID extraction node handles empty URLs gracefully, passing through the candidate data. The AI is instructed to evaluate based on form fields alone.
AI returns invalid JSON	The Parse AI Result node wraps JSON.parse() in a try/catch. On failure, it assigns a default 'average' classification and adds a note for manual review.
Duplicate emails	The 'Email Sent' column acts as an idempotency flag. Once set to 'Yes', the hourly email workflow will never process that candidate again.
Empty sheet run	The Has Pending Emails? IF node exits the false branch (does nothing) when the master sheet has no rows with Email Sent = No.

Google API rate limits

Both workflows run on slow schedules (30 min and 1 hour) to stay well within Google API quotas. Gemini free tier allows 15 requests/minute.

6.1 Master Sheet Setup

Create a new Google Sheet with a tab named 'Candidates' and add these column headers in row 1:

Name | Email | Role | Submitted At | Resume URL | AI Score | AI Classification | AI Summary | AI Strengths | AI Concerns | AI Recommendation | Status | Email Sent | Email Sent At | Processed At